Schaltplan aus der Netzliste

DigiTrainer, Datei `js/schaltplan.js` (Branch `schaltplan`), Stand 2. November 2026

Den Ablauf steuert `layoutFromWires()`, die die folgenden Schritte nacheinander aufruft. Am Ende steht ein Modell aus Symbolen, Linien und Punkten, das `svgOf()` als SVG zeichnet.

1. Bauteilkatalog: `catalog()`

Diese Funktion legt fest, was überhaupt ein Symbol werden kann. Sie liest die Gattertabellen des Simulators (`norGatesIC0`, `andGatesIC2` usw.) über `liveBinding()`. Fehlen sie, zum Beispiel im Test ohne Browser, nimmt sie die Ersatzliste `FALLBACK_GATES`. Jedes Gatter wird zu einem Symbol mit Pins und Rollen (A und B als Eingang, Y als Ausgang). Dazu kommen Blöcke für Zähler, ALU, ADC und Anzeige (`block()`) sowie „Klemmen“ (`term`): die Schalter S0 bis S7 mit Q und Q̄, CLK und CLK̄ und die LEDs D0 bis D7. `prefer` ist eine Wunschreihenfolge, damit S0 über S1 steht.

2. Netzliste aus den Kabeln: `netsFromWires()`

Die Kabel liegen als Liste von Paaren `from`/`to` vor (`wireConnections`). Die Funktion fasst mit einer Union-Find-Struktur alles zusammen, was leitend verbunden ist. `root()` sucht den Stellvertreter einer Gruppe und kürzt dabei die Wege ab, `join()` vereinigt zwei Gruppen. Die vier Buchsen einer Verteilerleiste `l0` bis `l4` werden vorab als verbunden behandelt. Netze mit `vcc`/`gnd` fliegen raus (`isSupply()`), weil die Versorgung im Logikplan unsichtbar bleiben soll. Ebenso fallen Einzelpins ohne Partner weg.

3. Nur benutzte Bauteile: `usedSymbols()`

Aus dem Katalog bleiben nur Symbole mit mindestens einem verdrahteten Pin, und auch von deren Pins nur die verdrahteten. `sizeOf()` bestimmt dann Breite und Höhe aus der Zahl der Ein- und Ausgänge.

4. Signalrichtung: `buildEdges()`

Jedes Netz wird in gerichtete Kanten zerlegt, eine Kante je Eingang, immer vom ersten Ausgang des Netzes aus. So entsteht ein Graph „Symbol treibt Symbol“. Verbindet eine Kante ein Gatter mit sich selbst, wird sie als `local` markiert.

5. Spalten: `assignLayers()`

Erst findet eine Tiefensuche (`visit()`) die Rückkopplungen, etwa beim RS-Flipflop, und markiert sie als `back`. Ohne diese Kanten ist der Graph kreisfrei, und jedes Symbol bekommt als Spalte die Länge des längsten Wegs von einem Eingang her. Das ist der Grund, warum der Signalfluss von links nach rechts läuft: Schalter links, dann Stufe für Stufe, LEDs rechts.

6. Reihenfolge innerhalb der Spalte: `orderLayers()`

Damit sich möglichst wenige Leitungen kreuzen, sortiert die Funktion jede Spalte nach dem Median der Positionen der Nachbarn (`bary()`). Das läuft fünfmal hin und her: von links nach rechts nach den Vorgängern, dann zurück nach den Nachfolgern. Das ist das klassische Sugiyama-Verfahren für geschichtete Graphen.

7. Platzieren: `place()` und `channelAfter()`

Jetzt bekommen die Symbole Koordinaten. Zwischen zwei Spalten bleibt ein Kanal für die Leitungen. `channelAfter()` macht ihn so breit, dass jede durchlaufende Leitung eine eigene Spur hat. `countOuter()` zählt vorher, wie viele Leitungen außen herum müssen. `clusterFanouts()` rückt Ziele desselben Signals zusammen, damit Abzweige kurz bleiben. `layoutPins()` setzt danach die Pins an die Symbolkanten.

8. Symbolform: `gateOutline()` und `iecMark()`

Standardmäßig werden Rechtecke nach IEC 60617 gezeichnet, mit „&“, „≥1“, „=1“ oder „1“ und einem Kreis am negierten Ausgang. Ohne den IEC-Haken kommen die amerikanischen Formsymbole. `xorFront()` sorgt dafür, dass die Leitung beim XOR an der äußeren Kurve ansetzt.

9. Leitungen führen

Hier stecken die meisten Zeilen.

- `bandPlan()` reserviert oben und unten am Blatt Bänder für Leitungen, die mehr als eine Spalte überspringen (`isFar()`).
- `routeForward()` zieht die Vorwärtsleitungen. Nachbarspalten werden mit rechtwinkligen Leitungen über eine Spur im Kanal verbunden (`drawOrtho()`, `channelSlot()`). Mehrere Ziele desselben Signals teilen sich eine senkrechte Stammleitung (`drawTree()`). Ferne Ziele laufen über eine Stammleitung im oberen oder unteren Band und zweigen erst an der Zielspalte ab (`drawFarTree()`). `goesBottom()` entscheidet anhand der Mitte (`columnMid()`), ob oben oder unten. `arrangeBundles()` und `bundleCrossings()` probieren Spurreihenfolgen aus, bis sich die Bündel möglichst wenig kreuzen.
- `routeFeedback()` führt die Rückkopplungen. Kurze Schleifen bleiben 14 Pixel neben dem eigenen Symbol, lange nutzen das Band.

10. Nacharbeit: `separateClose()` und `straighten()`

`separateClose()` sucht parallele Stücke verschiedener Netze, die näher als 14 Pixel liegen, und schiebt sie so lange auseinander, bis die neue Lage frei ist. Die Anschlüsse an den Pins bleiben dabei fest. `straighten()` ersetzt danach Umwege durch Bahnen mit weniger Knicken, aber nur, wenn die neue Bahn keine zusätzlichen Kreuzungen erzeugt (`pathClear()`) und Abstand zu Symbolen hält (`hitsSymbol()`, `hugsSymbol()`).

11. Verbindungspunkte: `junctionDots()`

Ein Punkt kommt nur dorthin, wo mindestens drei Richtungen desselben Netzes zusammentreffen. Eine Kreuzung ohne Punkt verbindet also nicht, wie im Schaltplan

üblich.

12. Zeichnen und Anzeigen

`svgOf()` baut aus dem Modell den SVG-Text, die Farben kommen aus `palette()` (dunkel für den Bildschirm, hell für den Druck). `mount()` hängt beim Laden den Knopf „Schaltplan“ und das Dialogfenster in die Seite. `showPlan()` zeigt erst die Wartemeldung an und rechnet dann. `cachedModel()` merkt sich das Ergebnis je Verdrahtung (`wireStamp()`) und Symbolart, sodass SVG-Export (`savePlan()`), Druck (`printPlan()`) und Umschalten nicht neu rechnen müssen.

> **Tipp zur Fehlersuche:** Meist ist Schritt 5 der Schlüssel. Steht ein Gatter in der falschen Spalte, liegt es fast immer daran, welche Kante die Tiefensuche als Rückkopplung erkannt hat.